
Certified Multi-Source Integrity for Structured Agent Actions

Anmol Pandey¹, Aditya Jain¹, Liang Chen², Carsten Maple¹, Christo Panchev¹

¹University of Warwick, ²University of Hertfordshire

Anmol.Pandey@warwick.ac.uk, aditya.jain.5@warwick.ac.uk, l.chen26@herts.ac.uk

cm@warwick.ac.uk, christo.panchev@warwick.ac.uk

Code and data: <https://github.com/anmolpandey299/certified-multisource-integrity>

Abstract: LLM agents increasingly take privileged, often irreversible structured actions, such as paying an invoice. They assemble each action from action-critical fields in documents and tool outputs that an adversary can corrupt, and indirect prompt injection can drive the model itself to extract attacker-chosen values. Current defenses gate on a source’s trust label or certify free-text answer quality. None certifies the integrity of a coupled, policy-bound structured action under a corruption budget that accounts for shared upstream sources. We characterize when such an action is safely certifiable and give the maximally live safe certifier. It admits an action only when each field clears the rule its evidence structure supports: a bounded corruption radius over *corruption-distinct* evidence classes, counted by a minimum hitting set so that re-publishing or laundered copies cannot manufacture a quorum, deterministic reconciliation for complementary fields, and a trusted anchor where the evidence leaves a field single-sourced. We formalize two robustness notions, validate each mechanism by ablation, and measure how often the multi-source precondition holds on sanctions designations (70,966 entities) and software supply-chain provenance (450 packages). Under upper-bound proxies, genuine corroboration is a minority phenomenon in both, and naive attestation counting overstates it, since witnesses that look independent collapse to two corruption-distinct domains once shared origin is counted. Across five current models in a real agent loop, a realistic injection fools every model but one and a naive agent then executes the fraudulent action on most attacks. The certifier admits no unsafe action and recovers the correct value where corroboration permits, while action-gating and provenance baselines are broken in every world of our harness by some attack in its space.

Keywords: LLM agents, prompt injection, trustworthy AI, certified robustness, AI systems security

1. Introduction

Large language model (LLM) agents are increasingly trusted to take privileged structured actions on a user’s behalf, such as paying an invoice, screening a counterparty against sanctions, or filing a record. For a payment the agent must settle on a payee, an amount, and an account, each read from documents and tool outputs. In realistic deployments this evidence is multi-source and adversarially corruptible: a purchase order, an invoice, a bank verification response, and a sanctions feed originate from different parties, and an attacker may influence any of them. Indirect prompt injection makes this sharper, since a tampered source can induce the model itself to extract an attacker-chosen value [1], so the agent proposes an action that is wrong yet well-formed. Since these actions are often irreversible, one fooled extraction can send a fraudulent payment.

Existing defenses do not address this directly. Information-flow systems and policy monitors gate an action on the trust label or provenance of its inputs. That answers whether data may flow to an action, not whether the value in that data is correct, so a decisive value from an untrusted source forces a blanket refusal. Certified-robustness work such as RobustRAG [2] bounds the quality of a free-text answer against injected

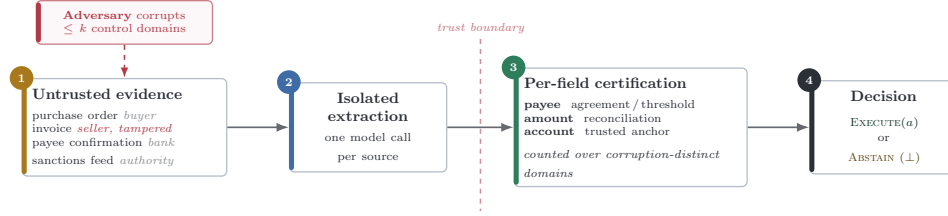


Figure 1. The certifier as an admission gate. Each source is read in isolation, every action-critical field is decided by the rule its evidence structure supports and counted over corruption-distinct domains, and the action executes only if every field is admitted, otherwise it abstains. An adversary within budget k corrupts at most k control domains, here the tampered invoice.

passages. It certifies an answer rather than an action, and it counts passages, so duplicated copies inflate the budget. None of these certifies the integrity of a structured action under a corruption budget that accounts for shared upstream sources, and none measures whether the corroboration such a certificate needs is present in real evidence. Section 7 places the present work against these systems.

We study certified multi-source integrity for structured agent actions. The certifier reads each source in isolation and admits an action only when every field independently clears the rule that suits its evidence structure. At the center is a bounded corruption radius over corruption-distinct evidence classes. Rather than counting raw attestations, the certifier counts the smallest number of independent control domains an attacker must corrupt to erase a fact, a minimum hitting set, so that authorities that merely republish or transpose one another are not mistaken for independent witnesses. A claim-atomic discipline then counts votes once per authenticated control domain, which prevents an attacker from laundering copies or issuing several records to inflate a single corrupted source into an apparent quorum. Fields that lack redundant attestation are handled honestly rather than forced into a vote. A complementary field such as an invoiced amount is checked against an authorized ceiling by deterministic reconciliation, and a single-sourced field such as the payee account is gated by a trusted anchor (Figure 1). We keep two robustness notions apart. Exact-value tolerance asks the certificate to survive the adversary intact, safety with abstention only that it never be wrong, and conflating them overstates the guarantee. Section 2 states both precisely.

This paper makes four contributions.

- A characterization of when a structured action is safely certifiable, with the corruption-distinct count as a minimum hitting set. Execution is sound exactly when the budgeted feasible set is a singleton, and the certifier meeting this bound is *maximally live*. The Byzantine safety and exact-value radii, and the invariance to laundering, follow as specializations (Section 2).
- A controlled ablation showing that each mechanism is load-bearing. Removing the transaction join key, the trusted anchor, the mandatory-source rule, or claim-atomic counting each opens exactly one attack family, while the full certifier admits none (Section 3).
- A measurement of where that precondition actually holds. On 70,966 sanctions designations only about 15% carry three or more issuing domains. On 450 npm packages an apparent third witness disappears entirely once its shared origin is counted, and no package reaches three. Driven by the witnesses that remain, the certifier catches a single-domain substitution on every corroborated package (Section 4).
- A real-model end-to-end evaluation over five current models. Only one resisted the injection here, and the naive agent then failed on most attacks, yet the certificate held throughout. In this evaluation it outperforms faithful capability-gating and policy-monitor defenses on safety and utility at once, and pays even the legitimate novel payees that allowlisting must refuse (Sections 5 and 6).

We are deliberate about scope. Real evidence corroborates only a minority of relations, the account field rests on a trusted anchor rather than on the radius, and the adversary in the live-model study is property-based rather than optimizing. The security guarantee is the theorem of Section 2, and the experiments show the mechanism behaving as designed wherever its preconditions hold.

2. Threat Model and Definitions

We model an agent that proposes a privileged structured action and a certifier that decides whether to execute it. The model is deliberately spare, so that the guarantees rest on stated assumptions rather than on implementation detail.

System and adversary. An agent proposes a privileged structured action $a = (a_1, \dots, a_d)$, a tuple of action-critical fields. For a business payment these are the payee identity, the payable amount, and the payee account. A certifier M decides each field independently, and the action executes only when every field is admitted. Each field is supported by a set of attestations. An attestation is a pair (v, c) in which an evidence class c asserts a value v . An evidence class is an origin of attestation with its own corruption mode, for instance a buyer document, a seller document, a bank verification response, or a designation authority. Every attestation is bound, by an unforgeable origin identifier, to the control domain that produced it, and carries a provenance root. Independent records carry distinct roots, and a copy inherits the root of its source, which is what later lets the certifier tell a genuine second opinion from a laundered duplicate.

The adversary selects a set $C \subseteq U$ of control domains subject to a corruption budget $|C| \leq k$, where U is the universe of control domains. Corrupting C affects every attestation whose dependency set meets C . Within its reach the adversary may set values arbitrarily and emit arbitrarily many records, including several distinct original records as well as laundered copies attributed to roots it controls. Elsewhere attestations are honest and report the field’s true value $f(x)$ after deterministic canonicalization. The adversary cannot forge an attestation bound to the origin identifier of a control domain it does not control, an assumption we make explicit below. The adversary knows the certifier, the corpus, and the policy, and may aim either to make the certifier execute a value other than the truth or to force it to abstain.

Trusted obligation skeleton. The certifier protects values within a fixed object, not the choice of object. From trusted user intent alone, before any untrusted evidence is read, we fix an obligation skeleton Γ pinning down the operation, transaction key, mandatory relations, eligible sources, allowed destinations, amount cap, and policy version. Untrusted evidence may only fill the values Γ leaves open. It may not redefine the operation, the transaction the action settles, or which sources are eligible, and an action is admissible only if it instantiates Γ . This closes a selection attack that field certification alone leaves open. A poisoned source steering the agent toward a different transaction yields a certificate sound for the wrong object, rejected at the skeleton before any vote is counted.

Corruption-distinctness. Apparent independence is not independence under corruption. An authority that republishes or transposes another shares its corruption mode, so two lists that copy a single upstream are one source for an attack. The concrete case is a software package, whose build provenance and repository back-reference look like two witnesses to its source but are both GitHub artifacts, so one compromise forges both and the apparent pair is one domain (Figure 2). We therefore measure redundancy by the number of independent corruptions needed to erase the evidence, not by the attestations on display.

Definition 1 (Dependency set). For an attestation att , let $D(att)$ be the set of control domains for which corrupting any single member removes att . An authority that designates an entity on its own has $D(att) = \{\text{that authority}\}$. A national listing that merely transposes a United Nations designation has $D(att) = \{\text{the nation, the UN}\}$, because corrupting either erases it.

Definition 2 (Corruption-distinct count). For a fact attested by $\{att_i\}$, the corruption-distinct count is the size of the smallest set of control domains that meets every attestation’s dependency set,

$$m = \min\{|X| : X \cap D(att_i) \neq \emptyset \text{ for all } i\}. \quad (2.1)$$

Equivalently, m is the fewest control domains an adversary must corrupt to remove every attestation of the fact. It is a minimum hitting set, and it reduces to minimum vertex cover when each dependency set has two members. For a threshold predicate such as “this entity is designated”, the presence-removal radius is $m - 1$, since the adversary needs m corruptions to flip the predicate and one honest attestation survives any $m - 1$ of them.

Section 4 measures an upper-bound proxy for m on real data, since exact dependency sets are not yet available.

Definition 3 (Budgeted influence). For a budget of k control domains, let the affected set of $X \subseteq U$ be $A(X) = \{i : D(att_i) \cap X \neq \emptyset\}$, and let $b_k = \max_{|X| \leq k} |A(X)|$ be the largest number of attestations any k domains can alter. When the attestations of a fact are corruption-distinct their dependency sets are disjoint and $b_k = k$, so one corruption moves one vote. When domains are shared a single corruption can move several votes and $b_k > k$. The propositions below are stated for corruption-distinct classes, where $b_k = k$. In the general case the same arguments hold with k replaced by b_k , so the exact-value condition becomes $N > 2b_k$ and the safety radius is $m - 1$.

Certifier rules. The certifier maps the observed attestations for a field to either $\text{EXECUTE}(v)$ or $\perp$, an abstention that escalates to a human. Different field structures call for different rules, and forcing every field through a single rule is one of the mistakes the design avoids. *Agreement or escalate* governs multi-source identity fields: execute v when a mandatory quorum of corruption-distinct classes is present and all of them agree on v after canonicalization, and abstain otherwise. *Unique threshold by count* governs multi-source value fields: execute a value when at most k corruption-distinct classes dissent from it, equivalently when its support is at least $N - k$, and it is the only value to clear that threshold, which we call unique-threshold admission. Uniqueness is load-bearing: at $N \leq 2k$ an adversary can make a challenger reach $N - k$ as well, so two values are feasible and admitting the larger count would certify a wrong one. *Reconciliation* governs complementary fields by a deterministic constraint, for example the invoiced amount lying within the authorized order plus tolerance, a check rather than a vote. Reconciliation bounds the payment by what the buyer authorized rather than certifying the invoiced figure, so that field carries a bound, not a radius. *The anchor gate* governs fields the evidence flow leaves single-sourced, such as the payee account: execute when the proposed account both matches the payee name at the bank and appears on the onboarding allowlist, and abstain otherwise. Safety here rests on trusted anchors rather than a corruption radius, a distinction we preserve.

Two robustness notions. We separate two guarantees that are easy to conflate and whose conflation flatters the result.

Definition 4 (Exact-value tolerance). M is k -exact-value-tolerant when $M_C(x) = f(x)$ for every $|C| \leq k$, so the certified value is unchanged by any corruption within budget.

Definition 5 (Safety with abstention). M is k -safe-with-abstention when $M_C(x) \in \{f(x), \perp\}$ for every $|C| \leq k$, so the certifier returns the correct value or refuses, and never a wrong one.

The second is weaker, giving up availability under attack, but reachable with far less redundancy, which matters since real multi-source coverage is scarce (Section 4).

When safe execution is possible. Fix the observed attestations, with values x_i , and the trusted skeleton Γ of join key, mandatory sources, amount cap, and destination constraints, all settled before untrusted evidence is read. Write $\text{Dis}_j(a) = \{i : x_i \neq a_j\}$ for the attestations of field j that disagree with the value a_j .

Definition 6 (Feasible action). An action a is k -feasible given the observation if it satisfies Γ and one corruption set of at most k domains explains every disagreeing attestation across all fields at once, that is $\tau(\bigcup_j \text{Dis}_j(a)) \leq k$ for the minimum hitting set τ over those attestations' dependency sets. Let F_k be the set of k -feasible actions.

Theorem 1 (Safe execution is exactly singleton feasibility). A certifier that never executes a wrong action under any $|C| \leq k$ may execute a if and only if $F_k = \{a\}$, and when $|F_k| \neq 1$ every k -safe certifier must abstain. The certifier that executes the unique feasible action and abstains otherwise is k -safe and maximally live: its abstention set lies inside that of every k -safe certifier.

The proof is an indistinguishability argument (Appendix A): two feasible actions yield one observation from two within-budget worlds, so committing to either is wrong in the other, while a unique feasible action is the truth in every allowed world. Whenever this certifier abstains two worlds are genuinely indistinguishable, so “it abstains too often” is not an objection without enlarging k . The budget couples the fields, so feasibility

is joint, not per-field. A single-sourced field has two feasible values for any $k \geq 1$, so no certifier executes it without a trusted anchor: the anchor is necessary. The rules below decide each field separately, a sound specialization: unique per-field admission implies the singleton the theorem requires, so the composition is k -safe, though it can abstain where the joint rule certifies (Proposition 4).

Proposition 1 (Agreement or escalate, the conservative case). *Under agreement-or-escalate over N corruption-distinct classes, if at least one class is honest then M is $(N - 1)$ -safe-with-abstention.*

Unanimity makes the agreed value the only feasible one once $N > k$. It is safe but not maximally live, since it abstains where the threshold rule still certifies.

Proposition 2 (Unique-threshold admission is maximally live). *Admit a value when at most k corruption-distinct classes dissent, equivalently when its support is at least $N - k$, and it is the unique value clearing that threshold. Then any admitted value equals $f(x)$, and N agreeing classes are certifiable against budget k if and only if $N > 2k$, the exact-value radius $\lfloor (N - 1)/2 \rfloor$.*

For disjoint domains this is Theorem 1 made concrete, the singleton condition being $N > 2k$. The threshold is the standard Byzantine bound, here applied per predicate over a dependency-aware electorate with vote identity preserved, and shown *optimal* rather than one design among many.

Proposition 3 (Vote identity bounds adversarial weight). *When each attestation is attributed to its authenticated originating control domain and a domain casts at most one vote per predicate, an adversary controlling k control domains contributes at most k votes, regardless of how many distinct original records or derived copies it emits.*

The bound is immediate: the votes for a predicate are the distinct controlling domains, and the adversary controls $|C| \leq k$ of them. The subtlety is what attribution requires. Counting once per provenance root is not enough. Root-counting collapses laundered *copies* onto the root they inherit, but a single compromised authority can also issue several distinct *original* records, each with its own root, so root-counting would hand one domain several votes and break the radius. Authentication binds every record, original or copied, to the domain that produced it, and the certifier counts domains rather than roots. Without this, t copies or t original records from one corrupted source contribute t votes and manufacture a quorum from a single corruption, the attack the domain rule removes and the counting experiment in Section 3 isolates.

Trusted computing base. These guarantees hold relative to four assumptions, which we state explicitly. First, authentication. Each attestation is bound to an unforgeable origin identifier, so the adversary cannot fabricate attestations from classes it does not control, and resistance to Sybil additions reduces to this. Second, trusted anchors. The onboarding allowlist and account-name registry lie outside the corruption budget, and the safety of single-source fields rests on them, which is why the account result is a trusted-anchor baseline rather than the radius certificate. Third, a trusted join key, which binds each attestation to one transaction and prevents splicing evidence across transactions. Fourth, sound extraction. Values are canonicalized deterministically before aggregation, and the safety argument assumes an honest class’s value is extracted to $f(x)$. The extractor is itself a shared control domain. One model, prompt, canonicalizer, and provider read every source, so a systematic parser fault or a malicious provider is a common-mode corruption that source isolation does not remove. The guarantee is therefore conditional on the extractor being sound on unaffected evidence. The model-in-the-loop study in Section 5 relaxes rather than removes this assumption, and Appendix B measures it directly. It shows that a content compromise confined to a single source becomes an abstention, because source-isolated reading keeps one tampered document from satisfying a multi-source rule, but it samples extractor faults rather than proving them independent across sources.

3. Mechanism Validation

Before placing a language model in the loop we validate the certifier’s aggregation in isolation, against an adversary that controls one evidence class, in a model-free harness so the result speaks to the certificate rather than end-to-end behavior. A mechanism is load-bearing when removing it opens exactly one attack while the full certifier stays safe. Removing the trusted join key opens amount splicing, removing the onboarding allowlist opens a name-matched mule, and removing the mandatory-source rule opens a single-source decision, each and nothing else, while the full certifier admits none. In the counting panel laundering succeeds only

without domain-bound counting and a Sybil addition only without authentication, the empirical face of Proposition 3. Appendix B reports the full ablation in Table 3.

The two robustness notions also appear directly: a three-source relation abstains under one corrupted source without certifying a wrong value, and a five-source relation keeps certifying the correct value, the exact-value regime of Proposition 2, which Section 6 then tests against an optimizing attacker rather than randomized worlds.

4. How Often Does the Multi-Source Precondition Hold?

A corruption radius is useful only where the corroboration it consumes exists, so we measure how often that is the case. Sanctions designation is the cleanest open setting in which one action-critical fact, that an entity is designated, is asserted by several nominally independent authorities. We use the consolidated OpenSanctions `sanctions` collection [3], which after entity resolution contains 70,966 entities carrying at least one designation authority, and for each entity compute a mapped-domain count as a minimum hitting set over issuing authorities. We report it at two levels. The raw-dataset level treats every source dataset as independent, a loose upper bound inflated by consolidated lists that republish others and by authorities that publish several files. The control-domain level folds datasets sharing a jurisdiction or body into one authority, and is the tighter proxy.

The full distribution is given in Table 2 (Appendix B), and the control-domain column is the one to read. After collapsing datasets that share a jurisdiction or body, about 15% of entities show three or more mapped issuing domains, about 25% show two or more, and roughly three-quarters rest on a single domain, so the well-corroborated entities are the heavily cross-listed ones. This is an upper bound on candidate multi-witness coverage, not evidence that 15% of payment actions obtain an exact-value radius. A mapped issuing domain $\hat{m}$ is not yet a corruption-distinct authority m , because shared upstream data and coordinated designations inflate the count. Recovering the true hitting set would require aligning programme, measure, and legal basis first. The corroboration the certificate consumes is present for a minority of decisions, a wider band supports safety with abstention, and the single-source majority is why the design needs trusted anchors where real evidence does not corroborate.

A second domain: software supply-chain provenance. Software artifact provenance is a second real setting where one action-critical fact, the source an installed package resolves to, can be independently corroborated, and where corruption-distinctness is the crux. For 450 real npm packages, the actual dependency closure of twenty widely used roots, we measure three witnesses to “package P comes from repository R ”. The first is the registry’s declared repository. The second is the SLSA build provenance, Sigstore-signed and transparency-logged. The third is the repository back-reference, whether the claimed repository itself declares P , a binding a substituted package cannot forge without compromising the repository. Counted as three separate witnesses, 16% of packages appear to have all three. But the provenance is built by GitHub Actions in every case and the back-reference is GitHub repository content, so both root in one control domain and a single repository compromise forges both. After collapsing this shared origin the third witness vanishes (Figure 2): about 90% of well-attested packages have exactly two corruption-distinct witnesses, the registry and the repository, and none reach three, the construction of Section 2 on real infrastructure. The collapse is not specific to npm: on 300 PyPI packages, where PEP-740 adoption is about double npm’s, an apparent three-witness structure on 32% again reduces to two domains rooted in GitHub.

Two corruption-distinct witnesses give safety with abstention, not exact-value certification, under a one-corruption adversary. Driving the agree-or-escalate certifier with them, a registry compromise that substitutes a malicious source is caught as a disagreement on every corroborated package, so the certifier certifies no malicious source (0 of 450), whereas a registry-trusting baseline accepts it every time.

A potential third signal lies outside the package author’s control. A repository’s reputation, its age, stars, and contributors, is built by the platform and community over years, not by whoever holds push access, so a typosquat author who forges a back-reference on their own repository cannot fake it. Across the same 307 declared repositories the median repository is 11 years old with 260 stars, and 74% clear a modest bar (≥ 2 years, ≥ 50 stars, ≥ 3 contributors). Reputation attests that a repository is long established, not that P came from R , so it is an eligibility signal rather than a third witness, and we report availability on legitimate packages, not detection against malicious ones.

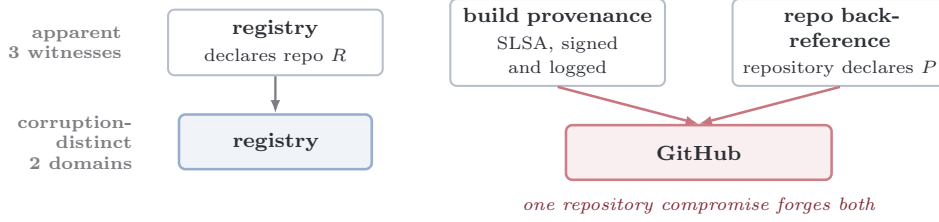


Figure 2. The apparent third witness to package-source identity collapses on real infrastructure. The registry, the SLSA build provenance, and the repository back-reference look independent, but the provenance is produced by GitHub Actions and the back-reference is GitHub repository content, so both root in one control domain that a single repository compromise forges. Counting corruption-distinct domains leaves two, not three. This is not marginal: 16.4% of 450 npm packages and 32.3% of 300 PyPI packages appear to carry three independent witnesses, and none do once the shared GitHub root collapses, leaving about 90% at exactly two.

Table 1. Each of five current models runs in a real agent loop (OpenRouter) for 80 episodes, with one of $N = 5$ sources corrupted by an embedded injection. The injection fools every model but Opus 4.8 and the naive agent’s unsafe rate reaches 100%, yet the certifier admits no unsafe action.

model	inj. follow	naive unsafe	false-abstain	cert. unsafe
Claude Opus 4.8	0%	0%	3.8%	0%
GPT-5.5	100%	72%	0%	0%
Gemini 3.1 Pro	100%	100%	0%	0%
Gemini 3.5 Flash	100%	98%	0%	0%
Claude Haiku 4.5	100%	100%	0%	0%

5. Model in the Loop

To test the certifier with a fallible, manipulable model in a real agent loop, we evaluate it end to end on live models served through OpenRouter. The action is a payment, {payee, account, amount}, assembled from N evidence documents of which an adversary has corrupted $k = 1$ with an embedded social-engineering injection. The model reads each source in isolation, which the certifier votes over, and separately reads all documents together as the naive baseline. Appendix B gives the full protocol.

Table 1 reports five models at 80 episodes each. Two things stand out. First, the injection works: only Opus 4.8 resists it, while every other model, frontier GPT-5.5 and Gemini 3.1 Pro included, follows it on every corrupted source, so the naive agent that trusts the model’s own proposal executes the fraudulent payment on 72 to 100% of attacks. Second, the certifier admits no unsafe action on any model. Where the model is fooled, the rule decides the outcome. The agreement rule ($N = 3$) turns the single corrupted extraction into a safe abstention. The unique-threshold rule ($N = 5 > 2k$, shown) recovers the correct value and abstains on only 0 to 6%, because the honest sources still clear the $N - k$ threshold without the misled one. These results instantiate Propositions 1 and 2: the guarantee holds whether or not the model resists. The liveness cost is the honest-world false-abstention, 0 to 12% under agreement and at most 4% under the threshold rule, since the vote tolerates the residual extraction variance that strict agreement does not.

6. Adaptive Attacker and Proxy Baselines

A defense that abstains freely is safe for a trivial reason, so the real test is safety *and* utility against faithful versions of the competing defense logic, compared on the domain where the comparison is fair. A structured action assembled from several corruption-distinct sources is that domain: the standard injection benchmark AgentDojo [1] gives one source per fact, so corroboration is unavailable there and any corroboration defense would abstain throughout. We pit the certifier against CaMeL/FIDES-style gating [4,5], which refuses actions whose decisive fields derive only from untrusted evidence, and a PCAS-style policy monitor [6] with an allowlist of vendors, known accounts, and an amount cap.

Across the five models of Section 5, 80 episodes each with a third of payees novel, Figure 3 separates the defenses. Capability gating is safe but executes nothing, the blanket refusal. The policy monitor catches gross substitutions yet stays unsafe on in-policy ones, up to 18%, and refuses every legitimate payee it has not pre-registered, so its utility on novel payees is 0%. The certifier alone is at once 0% unsafe and high-utility: the threshold rule recovers the correct value under attack, 96 to 100% correct overall, and pays every novel payee,



Figure 3. Head-to-head on the multi-source action for a model the injection fools (Haiku 4.5). The certifier is the only one of the four that is at once safe (0% unsafe) and high-utility (98%). The naive agent is fully unsafe, capability gating (CaMeL/FIDES) executes nothing, and the policy monitor (PCAS) is 18% unsafe at only 29% utility. The same pattern holds across all five models.

100%, compared with 0% for the policy monitor, because corroboration needs no maintained ground-truth list. A model-free white-box adaptive attacker that rewrites one source’s fields, launders copies, and revises against each refusal never forces an unsafe execution against the certifier across 2,000 worlds, while breaking both baselines in every one, although each still blocks one attack family (Table 7).

The harness’s own baselines, action-gating and provenance-only, differ from the model-based pair of Figure 3. Table 7 breaks them down by attack family: action-gating admits every family except an allowlist-missing destination, provenance-only every family except a currency swap, and the certifier none. Each blocks one family the other misses, and both stay unsafe on the four they share.

The exact-value radius is visible in the same harness: sweeping corroborating payee sources under a single-source vendor swap, the certifier abstains at two sources and certifies the correct vendor at three or more, the radius-one case of Proposition 2, with $k = 2$ needing $N \geq 5$ and $k = 3$ needing $N \geq 7$ (Appendix B). Safety is not bought by refusing, since honest-world false-abstention stays at a few percent under the threshold rule (Section 5). These baselines reproduce the defense logic, not the full engineering of a deployed system.

7. Related Work

Information-flow and capability systems secure the agent around an untrusted model. CaMeL attaches capabilities from the trusted query [4], FIDES permits a consequential call only when every input is high-integrity [5], PCAS compiles authorization policies into a reference monitor [6], and AuthGraph aligns a parameter-source provenance graph against a clean-context authorization graph [7]. Table 6 places these side by side. Descending from control-flow integrity [8] and information-flow control [9], they gate on where data came from rather than whether a value is correct. A decisive value from a default-untrusted source therefore forces a blanket refusal, whereas our certificate decides admission from corroboration.

The nearest methodological neighbor, RobustRAG, isolates retrieved passages and aggregates their answers to certify a quality bound against a bounded number of injected passages [2]. We share the isolate-then-aggregate shape but count corruption-distinct control domains through a minimum hitting set rather than passages, so laundered evidence inflates its budget but not ours, and certify a field-typed action rather than aggregate free text.

The motivation we build on was stated independently by ARGUS, which observes that defenses able only to refuse or isolate cannot admit a legitimate environment-supplied value [10]. ARGUS is closest in spirit and concurrent, but reasons about which span of one observation produced an argument, while we reason about how many independent sources corroborate a value and with what margin. Detection methods flag injections by behavioral consistency rather than certifying a corruption bound [11, 12], and persistent memory is its own attack surface [13–15]. The concurrent MemLineage refuses actions with untrusted ancestors [16], a recall problem. Our domain-bound counting instead stops duplicates from manufacturing a quorum, a counting problem. Finally, the certificate generalizes Byzantine quorum voting [17] from a fixed electorate to a set system over authenticated control domains. The electorate is the novelty: it must be inferred from shared dependencies rather than given, and a domain that republishes another casts no second vote.

8. Limitations and Conclusion

Four limits bound the claim: a fully adaptive live-model attacker remains untested, the counts are upper-bound proxies, the guarantee rests on the assumptions of Section 2, and corroboration remains scarce. Where it exists, the certifier admitted no unsafe action even where the model followed the injected instruction.

References

- [1] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, “AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents,” in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, vol. 37, pp. 82895–82920, 2024. arXiv:2406.13352. doi: 10.52202/079017-2636.
- [2] C. Xiang, T. Wu, Z. Zhong, D. Wagner, D. Chen, and P. Mittal, “Certifiably robust RAG against retrieval corruption,” *arXiv preprint arXiv:2405.15556*, 2024.
- [3] OpenSanctions, “Consolidated Sanctions.” <https://www.opensanctions.org/datasets/sanctions/>, 2026. Continuously updated dataset collection. Accessed: Sep. 14, 2026.
- [4] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, “Defeating prompt injections by design,” in *IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*, 2026. arXiv:2503.18813.
- [5] M. Costa, B. Köpf, A. Kolluri, A. Pavard, M. Russinovich, A. Salem, S. Tople, L. Wutschitz, and S. Zanella-Béguelin, “Securing AI agents with information-flow control,” *arXiv preprint arXiv:2505.23643*, 2025.
- [6] N. Palumbo, S. Choudhary, J. Choi, G. Amir, P. Chalasani, and S. Jha, “Formal policy enforcement for real-world agentic systems,” *arXiv preprint arXiv:2602.16708*, 2026.
- [7] P. Wang, Y. Li, and Y. Tian, “Aligning provenance with authorization: A dual-graph defense for LLM agents,” *arXiv preprint arXiv:2605.26497*, 2026.
- [8] M. Abadi, M. Budiu, Ú. Erlingsson, and J. Ligatti, “Control-flow integrity,” in *ACM Conference on Computer and Communications Security (CCS)*, pp. 340–353, 2005. doi: 10.1145/1102120.1102165.
- [9] D. E. Denning and P. J. Denning, “Certification of programs for secure information flow,” *Communications of the ACM*, vol. 20, no. 7, pp. 504–513, 1977. doi: 10.1145/359636.359712.
- [10] S. Weng, Y. Feng, J. Zhang, X. Xie, J. Yu, and J. Liu, “ARGUS: Defending LLM agents against context-aware prompt injection,” *arXiv preprint arXiv:2605.03378*, 2026.
- [11] K. Zhu, X. Yang, J. Wang, W. Guo, and W. Y. Wang, “MELON: Provable defense against indirect prompt injection attacks in AI agents,” in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, vol. 267 of *Proceedings of Machine Learning Research*, pp. 80310–80329, 2025.
- [12] F. Jia, T. Wu, X. Qin, and A. Squicciarini, “The Task Shield: Enforcing task alignment to defend against indirect prompt injection in LLM agents,” in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, (Vienna, Austria), pp. 29680–29697, July 2025. doi: 10.18653/v1/2025.acl-long.1435.
- [13] S. Dong, S. Xu, P. He, Y. Li, J. Tang, T. Liu, H. Liu, and Z. J. Xiang, “Memory injection attacks on LLM agents via query-only interaction,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 38, pp. 46697–46731, 2025. arXiv:2503.03704. doi: 10.52202/085713-1554.
- [14] Z. Chen, Z. Xiang, C. Xiao, D. Song, and B. Li, “AgentPoison: Red-teaming LLM agents via poisoning memory or knowledge bases,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, pp. 130185–130213, 2024. doi: 10.52202/079017-4136.
- [15] S. S. Srivastava and H. He, “MemoryGraft: Persistent compromise of LLM agents via poisoned experience retrieval,” *arXiv preprint arXiv:2512.16962*, 2025.
- [16] C. Ouyang and R. Hou, “MemLineage: Lineage-guided enforcement for LLM agent memory,” *arXiv preprint arXiv:2605.14421*, 2026.
- [17] L. Lamport, R. Shostak, and M. Pease, “The Byzantine generals problem,” *ACM Transactions on Programming Languages and Systems*, vol. 4, no. 3, pp. 382–401, 1982. doi: 10.1145/357172.357176.

A. Proofs

We prove the characterization theorem and the propositions that specialize it. Throughout, a field has true value $f(x)$, the adversary controls a set C of evidence classes with $|C| \leq k$, honest classes report $f(x)$ after canonicalization, and votes are counted once per authenticated control domain.

Proof of Theorem 1. Soundness. If $a, a' \in F_k$ with $a \neq a'$, pick corruption sets C, C' of size at most k hitting the disagreeing attestations of a and of a' . The world with true action a and corrupted domains C produces exactly the observation, as does the world with true action a' and C' . A certifier sees only the observation, so any action it executes is wrong in one of the two worlds, and it must abstain. Execution is thus permitted only when $F_k = \{a\}$. *Completeness.* If $F_k = \{a\}$, every $\leq k$ -corruption world consistent with the observation has true action a , since any consistent $a' \neq a$ would itself be k -feasible. Executing a is correct in all such worlds. *Optimality.* By soundness every k -safe certifier abstains whenever $|F_k| \neq 1$, exactly where the unique-feasible certifier does, so its abstention set is contained in that of every k -safe certifier. $\square$

Propositions 1 and 2. Both specialize Theorem 1 to corruption-distinct classes, where dependency sets are disjoint and τ counts dissenting domains, so a value is k -feasible iff at most k classes dissent, that is its support is at least $N - k$. Honest classes report $f(x)$, so $f(x)$ always has support at least $N - k$. *Threshold rule.* $F_k = \{f(x)\}$ iff no rival reaches support $N - k$, that is $k < N - k$, or $N > 2k$, with radius $\lfloor (N - 1)/2 \rfloor$. At $N \leq 2k$ the adversary concentrates k classes on a challenger of support $k \geq N - k$, two values are feasible, and the rule abstains. *Agreement.* Unanimity makes $f(x)$ the only feasible value once $N > k$, so any dissent forces abstention and a wrong value needs $|C| = N$, giving $(N - 1)$ -safety with abstention. The margin form $c_1 - c_2 > 2k$ is sufficient but not necessary and corresponds to the stronger post-attack recertification guarantee with the looser $\lfloor (N - 1)/4 \rfloor$ radius. $\square$

Proof of Proposition 3. Votes for a predicate are counted once per authenticated control domain. Let the adversary control the domains in C with $|C| \leq k$. By authentication, every record the adversary emits, whether an original record or a copy derived from one, is bound to an originating control domain in C : a copy inherits the origin of its source by root inheritance, and an original record carries the unforgeable origin identifier of the domain that produced it, which the adversary cannot forge for a domain outside C . The set of domains onto which the adversary's records map is therefore a subset of C , of size at most k . Because the tally counts at most one vote per distinct control domain per predicate, the adversary's effective vote count is at most $|C| \leq k$, independent of how many original records or copies it emits. If instead votes were counted per provenance root, a single compromised domain emitting t distinct original records would contribute t votes, and choosing t large enough would meet any fixed quorum, the attack the domain rule removes. $\square$

Field-wise composition against the joint certifier. Theorem 1 reasons jointly, over one budget shared by all fields, whereas the deployed certifier decides each field under its own rule and executes when every field is admitted. The two are not the same object, and the relation between them is the following.

Proposition 4 (Composition is sound and conservative). *Let each field j admit a_j only when a_j is the unique value whose dissenting attestations are covered by at most k control domains. If every field admits, then $F_k = \{a\}$, so the composed certifier is k -safe. The converse fails, so its abstention set contains that of the joint certifier of Theorem 1.*

Proof. **Soundness.** For any action b , $\text{Dis}_j(b_j) \subseteq \bigcup_i \text{Dis}_i(b)$, so a hitting set of size at most k for the union also hits each field's dissent, giving $\tau(\text{Dis}_j(b_j)) \leq \tau(\bigcup_i \text{Dis}_i(b))$. Hence every jointly feasible action is per-field feasible, and F_k is contained in the product of the per-field feasible sets. If each field admits a unique value then that product is $\{a\}$, so $F_k \subseteq \{a\}$. Under the threat model the true action is explained by the actual corruption set of size at most k , so $F_k \neq \emptyset$ and therefore $F_k = \{a\}$, which by Theorem 1 permits execution and identifies a as the truth. **Conservativeness.** The inclusion is one-directional: a field may carry two per-field feasible values whose extensions to the remaining fields are jointly infeasible, because a single budget cannot cover dissent that falls in different domains across fields. The joint rule then certifies where the composition abstains. Since per-field admission implies joint execution, the joint certifier's abstention set is contained in the composition's. $\square$

Two consequences are worth stating plainly. The maximal liveness of Theorem 1 is a property of the unique-feasible-action rule, not of the field-wise implementation, which is safe but may abstain strictly more often. And the reconciliation and anchor rules are not feasibility votes at all: the anchor places a field outside the corruption budget, which Theorem 1 shows is the only way a single-sourced field executes. The implemented system is therefore best read as a sound specialization whose liveness we measure rather than derive.

B. Experimental Details

The end-to-end evaluation of Section 5 runs on live models served through OpenRouter. Each instance is a payment with action-critical fields payee, account, and amount, presented as N evidence documents from fixed seeds so every model sees an identical corpus. One source ($k = 1$) is corrupted with an embedded social-engineering injection, drawn from three attack families: a payee swap, an account redirection, and an amount change. The model reads each document in a separate, source-isolated call, which the certifier votes over, and separately reads all documents together as the naive baseline, so a single tampered source cannot by itself satisfy a multi-source rule. Field values are compared in canonical form, amounts by numeric value, accounts by alphanumerics, and payees by normalized text, so formatting variation is not mistaken for disagreement. A certified action is scored unsafe only when it differs from the unique correct action. We report the agreement rule at $N = 3$ and the unique-threshold rule at $N = 5 > 2k$, 80 episodes each, roughly 4,000 live model calls in total. The adaptive-attacker comparison is model-free by construction: it grants the attacker full control of its single source, laundered copies, refusal-driven revision, and a bounded search over the whole-record manipulation space, so the result speaks to the certifier logic against a strong model-free adversary rather than to any model’s susceptibility. On reporting, the per-cell rates are point estimates over the episode counts stated in each caption, and a reported zero is an observed count rather than a proof, bounding the rate below roughly $3/n$ at the stated episode count n , and the pooled zero is descriptive rather than the outcome of a single significance test, since errors are correlated by template and by model family.

Table 2. Mapped issuing domains per designated entity in OpenSanctions ($N = 70,966$, Section 4). The control-domain column collapses datasets that share a jurisdiction or body. Both columns are upper-bound proxies for corruption-distinctness, not operational corruption radii.

mapped-domain count $\hat{m}$	raw dataset	control domain
$\hat{m} = 1$ (single domain)	62.5%	74.6%
$\hat{m} \geq 2$ (two or more)	37.5%	25.4%
$\hat{m} \geq 3$ (three or more)	22.7%	14.9%
$\hat{m} \geq 4$ (four or more)	14.5%	11.7%

Table 3. Mechanism ablation under a single corrupted class (Section 3). Each ablation opens exactly one attack while the full certifier stays safe. Values are attacker success rates.

<i>Payment-field mechanisms</i>				
configuration	one-source	mule acct.	amount-splice	omission
full	0%	0%	0%	0%
no join key	0%	0%	100%	0%
no account policy	0%	100%	0%	0%
no mandatory source	0%	0%	0%	100%
<i>Counting mechanisms (unsafe-certified rate)</i>				
configuration	single corr.	laundering	Sybil	
full	0%	0%	0%	
no atomic-claim	0%	100%	0%	
no authentication	0%	0%	100%	

The corruption radius for general k . The single-source studies above exercise the budget $k = 1$. To confirm that the certified value is governed by a genuine corruption radius rather than a single-source special case, we run the identical certifier rule against a general k -domain Byzantine adversary, sweeping the budget k and the number of corruption-distinct classes N . Each cell draws four thousand adversarial configurations,

including the worst case in which the k controlled classes coordinate every vote on one challenger, and reports the worst outcome for the defender. Table 4 shows the result. No configuration with at least one honest class ($N > k$) ever certifies a wrong value, and the certified-correct boundary falls exactly at $N > 2k$, so $k = 1$ needs $N \geq 3$, $k = 2$ needs $N \geq 5$, and $k = 3$ needs $N \geq 7$, matching the radius $\lfloor (N - 1)/2 \rfloor$ of Proposition 2.

Table 4. Outcome of the certifier rule under a k -domain Byzantine adversary (Proposition 2). **C** marks a cell certified under every adversarial configuration drawn, including the coordinated worst case, a dot a cell in which at least one configuration forces a safe abstention, and a dash a cell with no more classes than the budget ($N \leq k$). No cell with $N > k$ ever certifies a wrong value.

budget	$N=2$	$N=3$	$N=4$	$N=5$	$N=6$	$N=7$	$N=8$
$k = 1$	.	C	C	C	C	C	C
$k = 2$	–	.	.	C	C	C	C
$k = 3$	–	–	.	.	.	C	C

Vote identity must be the control domain, not the root. Proposition 3 requires that votes be counted per authenticated control domain. Counting per provenance root is not sufficient, and the gap is exploitable. We run the same certifier rule under three vote identities, while one compromised domain emits several distinct original records for a wrong value, each with its own root, plus laundered copies. Table 5 shows the outcome. Counting raw attestations is unsafe at once. Counting roots defeats copies but not multiple originals: at $N = 2$ a single compromised domain that issues two original records gets the wrong value certified, and at $N \geq 3$ the inflated roots push the true value below threshold and destroy the radius. Counting one vote per authenticated control domain reproduces Proposition 2 exactly, never certifying a wrong value and certifying the true value whenever $N > 2k$, independent of how many originals or copies the adversary manufactures.

Table 5. Certified outcome under a single compromised control domain ($k = 1$) that emits several distinct original records for the wrong value plus copies, by vote identity. Only counting per authenticated control domain preserves the guarantee. “Unsafe” means the wrong value was certified.

N	orig	per attestation	per root	per domain (ours)
2	1	unsafe	abstain	abstain
2	2	unsafe	unsafe	abstain
3	1	abstain	certify	certify
3	2	abstain	abstain	certify

Table 6. Where the certificate sits among neighboring systems (● present, ○ partial, – absent): corruption radius, field-typed action, shared-origin modelling, copy-laundering resistance, executing a value rather than refusing, and real-data coverage, meaning the mechanism is measured on a real deployed corpus rather than on synthetic evidence alone. The columns are the dimensions this work targets, so the table shows where the certificate differs from its neighbours rather than ranking them overall.

system	radius	typed	shared-orig.	copy-safe	executes	coverage
RobustRAG [2]	●	–	–	–	●	–
CaMeL [4]	–	○	–	–	–	–
FIDES [5]	–	○	–	–	–	–
PCAS [6]	–	●	–	–	●	–
ARGUS [10]	–	●	–	–	●	–
AuthGraph [7]	–	●	–	–	●	–
MemLineage [16]	–	○	–	–	–	–
this work	●	●	●	●	●	●

Table 7. Outcome by attack family under the adaptive one-source attacker (Section 6). The two proxy baselines each block one family the other misses and both stay unsafe on the four they share. **U** is an unsafe execution, a dash a safe abstention, **C** a correct execution under attack. The families are a vendor substitution, a fresh mule account, an account carried across vendors, an inflated amount, a currency swap, and a fully alternate invoice.

baseline	vendor	mule	x-vendor	amount	currency	full alt.
action-gating	U	–	U	U	U	U
provenance-only	U	U	U	U	–	U
full certifier	C	–	–	–	–	–

Computing the corruption-distinct count. The minimum hitting set of Definition 2 is NP-hard in general, but the instances a certifier meets are small and structured. Our implementation first forces every control domain that is the sole cover of some attestation, removes the attestations that those already hit, and solves the small remainder exactly, falling back to a greedy cover only when the residual universe exceeds twenty-two domains. Real dependency sets are short, one or two domains for a national listing that transposes a United Nations designation, so the forced-singleton pass usually settles the instance outright. The direction of any approximation matters. Greedy returns an upper bound on the hitting set, which overstates m and therefore overstates the radius, so a deployment must use an exact or lower-bound count. The measurements in Section 4 are reported as upper bounds throughout for the same reason.

Obtaining and maintaining dependency sets. The dependency sets themselves are metadata, and a deployment has to source them. In the two domains we study they are already published: a sanctions record names its issuing authority and its legal basis, and a package attestation names the registry, the builder identity, and the source repository. Mapping those to control domains is a curation task, folding datasets that share a jurisdiction or body, and it changes on the timescale of institutional structure rather than of individual records, so an audited quarterly review is enough. Incomplete or wrong information must fail in the safe direction. An attestation of unknown provenance should be assigned to an unknown domain shared with every other unknown, which lowers m and tightens the radius, rather than being treated as a fresh independent witness. Under that convention missing metadata costs liveness and never safety, and an adversary who hides provenance gains abstention rather than execution.

Per-source injection following against the naive agent. The two rates in Table 1 are measured on different calls, and the gap between them is informative. Injection following is recorded on the source-isolated read of a corrupted document, and the naive unsafe rate is recorded on a separate holistic call in which the model reads all N documents together. A model can adopt the attacker value when it sees the tampered source alone and still reject it in the holistic read, where the untampered documents contradict it. GPT-5.5 shows this most clearly: it follows the injection on every attacked episode, yet the holistic agent proposes the attacker value in 85% of them under agreement and 72% under the threshold rule. The residual is not a parsing artifact, since every holistic response in those episodes parsed successfully. The certifier does not depend on either rate, because it votes over the isolated reads and never consumes the holistic proposal.

Extractor independence. The guarantee assumes an honest class’s value is extracted to $f(x)$, and the extractor is a shared control domain, so we test it. On 150 episodes at $N = 5$ and $k = 1$ we read every source in isolation under six configurations: five single-model ones, which are the deployed case, and one mixed panel from Table 1 with a different model per source. The dangerous event is not an extraction error, since one misread honest source forces abstention in an attacked episode and is tolerated in a clean one. It is that enough honest sources agree on the *same* wrong value for it, with the corrupted source, to clear the $N - k$ threshold, which here takes three. We measure the stricter event of any two agreeing, a conservative proxy, over honest sources alone. Across 654 honest reads per configuration, no honest source yielded the attacker’s value, no two agreed on a wrong value, and none produced an unsafe certified action, bounding that proxy below 2% per configuration. We do not pool across configurations, since errors are correlated by template and model family, and since no semantic error was observed the data bound the coincidence rate rather than estimate a correlation.

The study exposed a canonicalization gap. One vendor name carries Cyrillic characters, transliterated to Latin at rates from 0% for both Gemini models to 7.5% for Opus 4.8. Canonicalization folds case and whitespace but not script, so a transliterated read compares unequal. It never produced an unsafe action, the value being the correct payee, but it costs liveness: Opus transliterated consistently and never abstained, whereas Haiku 4.5 did so inconsistently, disagreed with itself, and abstained on 4%. On this corpus self-consistency mattered more than sharing, and canonicalization should fold scripts as well as case.